



Ayudantía 2

INF129 Introducción a la Programación

Sergio Cárcamo Naranjo

August 9, 2026

DI - Universidad Técnica Federico Santa María

1. Motivación
2. Decisiones (Control de Flujo)

Motivación

En la vida real no ejecutamos siempre la misma rutina de forma lineal; tomamos decisiones basadas en nuestro entorno y las condiciones actuales.

- *Si* llueve, llevo paraguas.
- *Si* tengo saldo en la tarjeta, realizo la compra.

En programación ocurre lo mismo: necesitamos que nuestros algoritmos sean capaces de evaluar datos, reaccionar y tomar caminos distintos según la información que reciben.

Decisiones (Control de Flujo)

El tipo de dato booleano

Para que un programa pueda tomar decisiones, Python utiliza variables de tipo **booleano** (`bool`).

- Son la representación digital de un estado lógico.
- Solo pueden asumir dos valores: `True` (Verdadero) o `False` (Falso).

Cuidado con la sintaxis!

En Python, la primera letra siempre debe ser mayúscula. Escribir `true`, `FALSE` o `TRUE` provocará un error.

El condicional if

Usamos el **condicional if** para ejecutar un bloque de código **solo si** la condición evaluada es verdadera.

```
llueve = True
if llueve:
    print("Llevando el paraguas...")
```

La Indentación es obligatoria

En Python, el bloque de código que pertenece al **if** **debe** llevar una sangría (4 espacios o Tab). Si no la incluyes, el programa no compilará.

Operadores relacionales

Nos permiten comparar valores entre sí. Su resultado siempre será un valor booleano (True o False).

- == : Igual a (¡No confundir con el = de asignación de variables!)
- != : Distinto de
- > : Mayor que
- < : Menor que
- >= : Mayor o igual que
- <= : Menor o igual que

Nos permiten unir o modificar múltiples expresiones booleanas en una sola decisión más compleja:

- **and** : Será True **solo si todas** las condiciones evaluadas son verdaderas.
- **or** : Será True si **al menos una** de las condiciones es verdadera.
- **not** : Invierte el estado lógico. Transforma True en False, y viceversa.

Múltiples condiciones: elif y else

¿Qué pasa si hay más de dos caminos posibles?

- `elif`: Se evalúa únicamente si la condición anterior fue falsa. **Puedes usar una cantidad ilimitada de elif!**
- `else`: Es el camino por defecto. Se ejecuta solo si **todas** las condiciones anteriores fallaron. Solo puede haber uno y va al final.

Tip: Cuando combinen muchos operadores lógicos en estas condiciones, acostúmbrense a usar **paréntesis** para definir el orden de prioridad, igual que en matemáticas.

Condiciones anidadas

Es simplemente colocar un `if` dentro del bloque de otro `if`. Requiere aplicar una doble indentación.

```
if usuario_logeado == True:
    if saldo_cuenta > 0:
        print("Permitir transacción")
```

Dependiendo del contexto, una condición anidada suele ser lógicamente equivalente a unir dos condiciones con un operador `and`:

```
if (usuario_logeado == True) and (saldo_cuenta > 0):
    print("Permitir transacción")
```

Ejemplo

```
precio_juego = 29990
saldo_steam = float(input("Ingrese su saldo: "))
es_oferta_verano = True

if es_oferta_verano:
    precio_juego = precio_juego * 0.8  # 20% descuento

if saldo_steam >= precio_juego:
    saldo_steam = saldo_steam - precio_juego
    print("Juego añadido a tu biblioteca!")
    print("Nuevo saldo disponible:", saldo_steam)
else:
    falta = precio_juego - saldo_steam
    print("Saldo insuficiente. Te faltan:", falta)
```

¿Dudas hasta aquí?